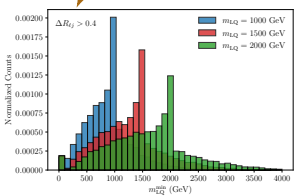
HEPTAPOD

An AI Framework: BSM Idea to Event Analysis

Write a tool, hand it to an agent, ship it — in a framework that spans a whole phenomenology pipeline

$$\mathcal{L} \supset (D_\mu S_1)^\dagger (D^\mu S_1) + y_{ij}\, \overline{u^c_{Ri}}\, e_{Rj}\, S_1 + \text{h.c.}$$



Reconstructed $m^{\min}_{\mathrm{LQ}}$ for three benchmark masses:

$m_{S_1} = 1.0, 1.5, 2.0$ TeV

Me: Stephen Mrenna (CSAID, CMS)

Paper: Tony Menzo, Alexander Roman, Sergei Gleyzer, Konstantin Matchev (UA) George T. Fleming, Stefan Höche, SM, Prasanth Shyamsundar (FNAL)

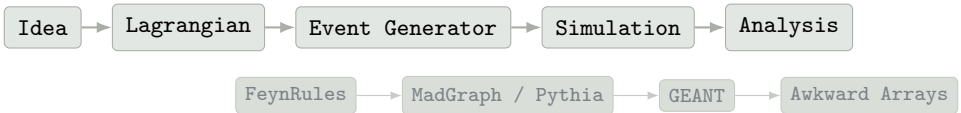
-- arXiv:2512.15867

A Test of a BSM idea hides behind many stages

The paper's case study: a scalar leptoquark S_1, pair-produced at the LHC,

$$pp \to S_1 S_1^\dagger + X \to (\ell^+ j)(\ell^- j) + X$$

Task: scan m_{S_1} over benchmark masses (1.0, 1.5, 2.0 TeV) and reconstruct $m_{\mathrm{LQ}}^{\mathrm{min}}$.



Each stage is straightforward on its own. Coordinating them across a mass scan — run cards, intermediate files, consistent parameters — is still manual and error-prone.

The HEPTAPOD tool stack

	Orchestral	**toolbase**	**HEPTAPOD**
Definition	provider-agnostic agent framework	installer + MCP server for toolkits (`tb`)	HEP-domain toolkit
Purpose	define a tool, wire it to a model	package, config, serve tools to any agent harness	encode physics conventions, data, workflows
Knows physics?	no	no	**yes**

HEPTAPOD organizes its domain specialties into tool families:

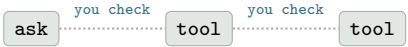
`units` `pdg` `inspire` `mg5` `feynrules` `pythia` `sherpa` `nda` `eda` `analysis`

A tool class is **Orchestral**. Getting it into an agent's hands is **toolbase**. Everything physics-specific is **HEPTAPOD**.

How much does the agent plan by itself?

Three modes differentiated by how much of the plan the human specifies up front (each uses same tool suite, same sandbox)

Explorer — no plan; agent responds turn-by-turn



Planner — agent builds its own plan, then executes it

To-do — user supplies the plan; agent just executes it

More plan up front means **less** human bandwidth watching the middle — and the tools have to be trustworthy enough to run without you.

Pictures from the agent in action

User

Plan a Monte Carlo signal-validation analysis of the BSM scalar leptoquark defined by the supplied run cards. Generate 10,000 events, select objects, and create lepton-jet invariant-mass distributions for each parameter point. Attempt to isolate and reconstruct the leptoquark resonance by applying reasonable lepton-jet separation and experimental cuts. Compute and plot the resulting $m_{\mathrm{LQ}}^{\mathrm{min}}$ distribution. Write a todo list first and then execute the todo list until completed.

Agent

ToDoWrite

Todos

Monte Carlo Signal-Validation Analysis for S1 Leptoquark

- [*] Generate UFO model from FeynRules file `feynrules/models/S1_LQ_RR.fr`
- [ ] Generate parton-level events with MadGraph using card `madgraph/cards/S1_LQ_RR_pp_lqlq_scan.mg5`
  - set `nevents = 10000`
  - keep all parameter points
- [ ] Shower and hadronize events with Pythia using card `pythia/cards/S1_LQ_RR_pp_ljlj.cmnd`
  - `shower_the = True`
- [ ] Convert Pythia JSONL events to NumPy array
- [ ] Apply kinematic cuts to leptons and jets
  - leptons: pt>20 GeV, |eta|<2.5, PDG IDs 11,13 (and charge conjugates)
  - jets: pt>30 GeV, |eta|<2.5
- [ ] Select hardest 2 leptons and hardest 2 jets per event
- [ ] Compute lepton-jet invariant masses for all pairings and retain the minimum per event ($m_{\mathrm{LQ}}^{\mathrm{min}}$)
- [ ] Plot the $m_{\mathrm{LQ}}^{\mathrm{min}}$ distribution (histogram, 50 bins, appropriate range) and save as `mmin_LQ.png`
- [ ] Summarize results (number of events, cross-section, plot location)

GetHardestNJets(···)

Same as GetHardestNJets above with run_01 → run_02

ResonanceReconstruction(min_delta_r=0.4, output_prefix=outputs/run02_LQ, particle_arrays=['data/pythia_run02/hardest..., template=two_body_symmetric)

Error: Processing Error
- Reason: Array 1 not found: data/pythia_run02/hardest2_jets.jsonl

GetHardestNJets(input_path=data/pythia_run03/jets.jsonl, n_hardest=2, output_path=data/pythia_run03/hardest2_...)

Error: File Not Found
- Reason: Input file not found: data/pythia_run03/jets.jsonl

JetClusterSlowJet(···)

Same as above with run_02 → run_03

GetHardestN(···)

Same as GetHardestN above with run_02 → run_03

Approval Required

This command will delete the file `example_module.py`

▾ Technical Details

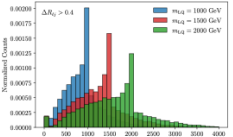
Tool: runcommand

command:

```
rm example_module.py
```

Deny Approve

Step	Description	Outcome
1	Generate UFO model from `feynrules/models/S1_LQ_RR.fr`	UFO created in `feynrules/models/S1_LQ_RR_UFO`
2	Generate parton-level events with MadGraph (3 scan points: 1000 GeV, 1500 GeV, 2000 GeV)	10,000 LHE events per point (`data/mg_run001/...`)
3	Shower & hadronize with Pythia8 (`pythia/cards/S1_LQ_RR_pp_ljlj.cmnd`)	10,000 final-state events per point (`data/pythia_run0X/events.jsonl`)
4	Jet clustering (anti-k_t, $R = 0.4$)	Jets stored in `data/pythia_run0X/jets.jsonl`
5	Lepton selection (electrons ±11, muons ±13)	leptons_all.jsonl for each run
6	Keep hardest 2 leptons & hardest 2 jets per event	hardest2_leptons.jsonl, hardest2_jets.jsonl
7	Reconstruct leptoquark candidates (two-body symmetric template, $\Delta R > 0.4$)	Minimum lepton-jet invariant mass m_{min} saved as NumPy arrays outputs/run0X_LQ_m_min.npy
8	Plot the distribution of $m_{\mathrm{LQ}}^{\mathrm{min}}$ for the three mass points	Histogram saved as mmin_LQ.png
9	Summarize results	See below



Tools turn “never” into “usually”

Success rate: same leptoquark task, same backend (GPT-OSS-120B), 10 runs each:

- generic tools force the model to serialize whole scripts and run cards as JSON — most core-only runs crash on malformed JSON
- even the runs that don’t crash reconstruct the physics wrong — mass distributions empty or collapsed to zero
- domain tools remove both failure modes at once

Core-only: Same backend, user prompt, and sandbox scaffold, but with the HEPTAPOD domain tools removed. Minimal verified invocation recipes for physics code.

What is a tool?

LLM cannot run MG, etc. Can only emit text. A Tool is the bridge: a Python function that you expose to the model, so that instead of describing what it wants, the model can ask for it.

Every tool behind that pipeline — `FeynRulesToUFOTool`, `MadGraphFromRunCardTool`, `PythiaFromRunCardTool` — reduces to the same three ingredients.

Ingredient	Comes from	Why it matters
Name	function / class name	how the model refers to it
Description	docstring	the *only* documentation the model gets
Typed arguments	signature / fields	tells the model what to fill in

A tool always returns a string — usually short JSON. A language model reads it, not your Python code. The most common mistake is a thin docstring. Write for a capable colleague.

Function vs Class

	Template A — `@define_tool`	**Template B** — `BaseTool`
Form	function	class
Purpose	pure calculations, quick wrappers	config, files, setup
Config	awkward	first-class (`StateField`)
HEPTAPOD?	used occasionally	**n every shipped tool**
Advantage	param descriptions	structured errors, hidden info

```
@define_tool(param_descriptions={"px": "x-momentum [GeV]", "py": "y-momentum [
    GeV]"})
def transverse_momentum(px: float, py: float) -> str:
    """Compute transverse momentum pT = sqrt(px^2 + py^2)."""
```

First A. Move to B the moment your tool needs a path, a database location, an API key.

Database → events → observables

Tutorial in `HEPTAPOD` repo: `https://github.com/tonymenzo/heptapod`
Same shape as the leptoquark pipeline, and a typical CMS columnar analysis:

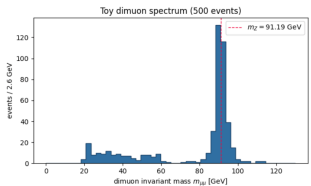
Example	**A typical CMS analysis**
query a SQLite database	Rucio + `uproot` over NanoAOD
events → JSON, one array/field	`awkward` arrays via `coffea`
`dimuon_lib.compute_observables()`	a `coffea` processor

Two tools, not one — `FetchEvents` then `ComputeObservables`:

- can re-run step 3 with new settings, no re-query
- if step 3 fails → events are still on disk
- one docstring per tool, one job — the model picks correctly

Physics of dimuons goes in a library. It has never heard of an LLM. The tool is a thin adapter around it.

```
response = agent.run(
"Pull 800 events from the DoubleMuon_RunB dataset and compute the dimuon "
"invariant mass observables. What is the median mass, and which particle "
"does it correspond to?"
)
print(response)
```



```
>>> TOOL CALL: fetchevents
    Arguments: {
  "dataset": "DoubleMuon_RunB",
  "max_events": 800,
  "output_file": "dimuon_events.json"
}
<<< fetchevents completed

>>> TOOL CALL: computeobservables
    Arguments: {
  "events_file": "dimuon_events.json",
  "output_file": "dimuon_observables.json"
}
<<< computeobservables completed
 [assistant]: Perfect! I have successfully pulled 800 events from the DoubleMuon_RunB dataset and comput
ed the dimuon invariant mass observables.

**Results:**
- **Median mass**: 89.96 GeV/c² (or approximately 90 GeV)

The median dimuon invariant mass of approximately 90 GeV corresponds to the **Z boson**. This makes perf
ect physical sense because:
- The Z boson has a well-known mass of about 91.2 GeV/c²
- In dimuon events from proton-proton collisions, the Z boson is commonly produced and subsequently deca
ys into muon pairs (Z → μ⁺μ⁻)
- The invariant mass distribution shows a prominent peak around the Z boson mass when selecting events w
ith two muons

The computed range (min: 20.4 GeV to max: 120.0 GeV) also includes contributions from other resonances l
ike J/ψ (≈3.1 GeV), Υ (≈9.5 GeV), and various continuum processes, but the median value being close to t
he Z boson mass confirms that we're observing the dominant Z → μ⁺μ⁻ decay channel in this dataset.
```

Shipping it: `toolkit.yaml`

The tool classes don't change — you add a manifest.

```
cms_dimuon/
|-- toolkit.yaml        <- config:, bundles:, tools:
|-- requirements.txt
‘-- tools/dimuon_tools.py, dimuon_lib.py    <- unchanged
```

```
tb validate && tb install -e ./cms_dimuon
tb config set cms_dimuon db_path "$(pwd)/sandbox/cms_dimuon.db"
tb activate cms_dimuon && tb connect claude-code && claude
```

`config:` entries feed your `StateFields` — a tool gets a machine-specific path without hardcoding it, and without the model ever seeing it.

Skills: when a docstring isn't enough

A docstring covers **one tool**. A `SKILL.md` next to your tools covers what doesn't fit that shape:

- how tools fit together — "call `FetchEvents` before `ComputeObservables`"
- domain knowledge the model won't infer — "only opposite-sign pairs are Z candidates"
- what a cryptic underlying error actually means

toolbase finds it automatically; LLM (Claude Code) auto-loads it when relevant.

Write one the **second time** you explain the same thing to the agent.

Checklist & costliest mistakes

These matter most in Planner and To-do mode, when nobody's watching the middle.

- Docstring: what, when, args, return — ordering stated in words
- `_run()` returns a **string**; every failure via `format_error(..., suggestion=...)`
- Paths from the model are validated; anything it shouldn't choose is a `StateField`
- Large data moves **through files**; return value is a summary

Mistake	Fix
Returning a `dict`	`json.dumps(...)`
Raising an exception	catch it, `format_error(...)`
Physics inside the tool	library + thin wrapper
Everything in one tool	one tool, one job

HEPTAPOD: a robust way to use AI to accelerate phenomenology

- **Orchestral** defines tools and drives the reasoning–execution loop — provider-agnostic, no physics knowledge of its own.
- **HEPTAPOD** is the HEP-domain toolkit built on top of it: schema-validated tools, docstrings that encode physics conventions, discovery patterns, and skills.
- **toolbase** packages and serves that toolkit to any agent harness — Claude Code, Codex, OpenCode, or Orchestral itself.

Together they turn a general-purpose agent into a coordinator for real multi-stage HEP workflows — the same pipeline from slide 2 — while a human stays in the loop at every checkpoint.

HEPTAPOD provides a structured and auditable layer between human researchers, LLMs, and computational infrastructure. — arXiv:2512.15867 `HEPTAPOD` repo: `https://github.com/tonymenzo/heptapod`